

Gender Prediction on Kaggle Dataset: Experimental Procedures and Team Implementation

Yun-Ting Lai
113423040
eleanor128@gmail.com

Wei-Siang Chen
113423052
nicolews.chen@gmail.com

ABSTRACT

This is a report for an in-class Kaggle competition, aiming to build a predictive model to determine the gender of respondents based on a dataset collected from an online survey. Both the training and testing datasets contain a significant number of missing values and invalid or extreme entries, making data preprocessing a crucial step in the pipeline. In this report, we detail our full experimental procedure, including data cleaning, feature engineering, model selection, hyperparameter tuning, and evaluation. We explored multiple feature combinations and designed several additional features such as BMI, keyword, and combo_keyword_bmi to capture subtle patterns in the data. We present two similar experimental approaches that both achieved a final accuracy of 86.38%. Detailed procedures, along with the reasoning behind each design choice, are described in the following sections. Techniques such as 10-fold cross-validation and KNN-based imputation were employed throughout the experiments.

Keywords

Kaggle Dataset, Gender Prediction, Data Preprocessing, Feature Engineering, K-Nearest Neighbors, KNN, RFE, Cross Validation, XGBoost.

1. INTRODUCTION

1.1 Task Description

This task aims to predict the gender of survey respondents based on their answers to a questionnaire. The dataset includes both numerical and categorical variables, such as height, weight, star sign, self-reported IQ, and self-introduction text.

The goal is to design a machine learning model that accurately identifies gender using the given features, some of which contain missing values, inconsistent formats, or noisy text. This challenge involves both data preprocessing and model design, with a final submission consisting of a CSV file with predicted genders for the testing set.

1.2 Data Overview

There are two datasets in this Kaggle challenge, the training dataset “boy or girl 2025 train_missingValue.csv” and the testing dataset “boy or girl 2025 test no ans_missingValue.csv.” Both the training and testing datasets contain 11 columns, including id, gender, star_sign, phone_os, height, weight, sleepiness, iq, fb_friends, yt, and self_intro. In the training dataset, the gender column is the target variable, where: 1 represents boy and 2 represents girl. In the testing dataset, the gender column is set to 0 for all entries, indicating that our goal is to predict these values. Except for id and gender, most columns contain missing values or invalid/extreme values, making proper data cleaning a crucial step before model training. A brief description of each column is provided below:

- *star_sign*: The responder's zodiac sign, written in Mandarin.

- *phone_os*: The responder's phone operating system, typically either iOS or Android.
- *height and weight*: Represented in centimeters and kilograms, respectively.
- *sleepiness*: A self-reported score ranging from 1 to 5, indicating how sleepy the responder was while answering the survey.
- *iq* and *fb_friends*: Self-estimated values provided by the responder.
- *yt*: The average number of hours per week the responder spends watching YouTube.
- *self_intro*: A word or a sentence written by the responder to describe themselves. Most of them written in English, few in Mandarin.

	Missing Values	Percentage	Data Type
id	0	0.0%	int64
gender	0	0.0%	int64
star_sign	86	20.3%	object
phone_os	78	18.4%	object
height	74	17.5%	float64
weight	85	20.1%	float64
sleepiness	91	21.5%	float64
iq	79	18.7%	float64
fb_friends	77	18.2%	float64
yt	90	21.3%	object
self_intro	104	24.6%	object

Figure 1. Overview of the training dataset

2. DATA PREPROCESSING

2.1 Feature Selecting

After a thorough examination of the dataset, we identified height and weight as the most important features for predicting gender. To validate this assumption, we conducted an initial experiment using only these two features. Missing values were filled using the mean values from the training set, and the resulting model achieved an accuracy of 82.15%. This result confirmed our intuition that features such as star_sign, iq, or yt might contribute less to the prediction task.

Additionally, we observed that the self_intro field could also provide meaningful information. This text field may include gender-indicative keywords such as “beautiful” or “handsome”, and its length might also differ between genders. Based on these insights, we decided to proceed with a simplified dataset containing only height, weight, and self_intro as our core features for further modeling and feature engineering.

2.1.1 Missing and Extreme Value in Height / Weight

We only tackle missing and extreme value in height and weight columns. To identify extreme outliers from the training dataset, we

defined acceptable ranges for the height and weight columns using the Interquartile Range (IQR) method.

Specifically, we calculated the first quartile (Q1) and third quartile (Q3) for both height and weight from the whole training dataset and determined the $IQR = Q3 - Q1$. We then defined the lower and upper bounds for valid values as:

$$Lower\ bound = Q1 - 3 \times IQR$$

$$Upper\ bound = Q3 + 3 \times IQR$$

This rule is more conservative than the typical $1.5 \times IQR$ threshold, allowing only extreme outliers to be flagged. After identifying and filtering out these outliers, we calculated the average height and weight for the entire training dataset, as well as for the male and female subgroups, using only the values within the acceptable range. These cleaned averages were then used to impute missing values in the corresponding features, ensuring statistical consistency while minimizing the influence of noise.

In addition to using height and weight directly, we also created a new feature called BMI (Body Mass Index) to better represent body composition. Similar to height and weight, we computed the average BMI for the entire training dataset, as well as separately for the male and female subgroups. These averages were later used to impute missing values. The standard formula is as below:

$$BMI = \frac{\text{weight (kg)}}{(\text{height (m)})^2}$$

Table 1. Average height, weight and BMI in Training Dataset

	Height (cm)	Weight (kg)	BMI
Boy	173.4	67.3	22.4
Girl	160.6	54.8	21.3
Whole Dataset	170.4	64.4	22.1

After obtaining the average values of height, weight, and BMI (both overall and by gender), we proceeded to clean the height and weight columns by replacing missing and extreme values. The steps are as follows:

- (1) **Convert negative values to positive:** If any height or weight values were negative, we first converted them to their absolute values.
- (2) **If both height and weight are missing:** Fill in both values using the average height and weight of the same gender.
- (3) **If both height and weight are extreme values (invalid):** Assign value **1** to both fields as a flag, since no reliable estimation could be made.
- (4) **If only height is invalid:** Fill in the value using the average height of the same gender.
- (5) **If only weight is invalid:** Fill in the value using the average weight of the same gender.
- (6) **If only height is missing:** Estimate height using the formula:

$$height = \sqrt{\frac{weight}{gender_BMI_average}} \times 100$$

- (7) **If only weight is missing:** Estimate weight using the formula:

$$weight = gender_BMI_average \times \left(\frac{height}{100}\right)^2$$

After replacing all missing and invalid values in the height and weight columns, we recalculated the BMI for each entry to ensure that every sample has a valid and consistent BMI value. This step guarantees that the BMI feature is complete and aligned with the imputed or corrected height and weight values.

2.2 K-Nearest Neighbors imputation model

The K-Nearest Neighbors (KNN) imputation method is a way to fill in missing values based on the distance between data points. The main idea is to find the K most similar samples from the dataset, using the other available features, and use their values to estimate the missing ones. Common distance measures include Minkowski Distance, Manhattan Distance, Cosine Distance, Jaccard Distance, Hamming Distance, and Euclidean Distance. Some studies suggest that Euclidean distance gives the best results when used for this purpose. KNN imputation works for both continuous and categorical data, and it helps keep the structure and relationships in the original data. Since the dataset in this study is relatively small, the computing cost is low [1]. The Euclidean distance formula is as below:

$$Dist_{xy} = \sqrt{\sum_{k=1}^m (X_{ik} - X_{jk})^2}$$

In this formula, $Dist_{xy}$ represents the Euclidean distance between two data instances. The variable k denotes the total number of data attributes or dimensions. For each attributes j (where $j = 1, 2, 3, \dots, k$), X_{ik} refers to the value of the j -th attribute in the instances with missing data, while X_{jk} represents the corresponding attribute value in the complete data record [1]. In this research, KNN imputation was only used for certain continuous columns.

In this study, missing values were filled using the K-Nearest Neighbors (KNN) imputation method, with the number of neighbors set to $k=8$. The imputation process was done in two steps. First, the KNN model was fitted on the training data using only the available values. Then, this same model was used to transform both the training and test datasets. Importantly, the test data was imputed based on the structure learned from the training data only, which helps prevent data leakage and ensures a fair evaluation.

2.3 Additional Feature Created

In addition to the BMI feature, we created five additional columns to enrich the feature set and capture more information from the self_intro text and user profiles. These newly added features include: self_intro_length, self_intro_word_count, keyword, combo_keyword_bmi and combo_keyword_height. The details of each new feature are as follows.

Both **self_intro_length** and **self_intro_word_count** were derived after preprocessing the self_intro field. The preprocessing steps included:

- Converting all text to lowercase
- Removing punctuation marks and special characters

After this cleaning process, self_intro_length was calculated as the number of characters, and self_intro_word_count was computed based on the number of whitespace-separated tokens.

The **keyword** column was created to capture the presence of gender-indicative words within the self_intro field. We study the training dataset and manually create two dictionaries:

- boy_keywords = ['handsome', 'man', 'cool', 'guy', 'gay', 'nerd', 'boy', 'dude', 'papa', '男', '紳']

- `girl_keywords = [ 'beautiful', 'cute', 'pretty', 'girl', 'beauty', 'adorable', '可愛', '女' ]`

If the `self_intro` field of a given sample contains any word from the `boy_keywords` dictionary, we assign the value 1 to the keyword column. Similarly, if it contains any word from the `girl_keywords` dictionary, we assign the value 2. If no keywords from either dictionary are found, the value 0 is assigned to indicate the absence of gender-related keywords. In cases where both boy and girl keywords appear in the same text, we assign value 2, though this condition is really rare. This simple keyword-based approach allows us to encode linguistic gender cues from user-written text in a structured form suitable for modeling.

The two `combo_keyword_*` columns represent the combination of the keyword feature with either BMI or height. This idea was inspired by our own human reasoning when attempting to predict gender — we naturally consider multiple attributes together, such as “a feminine keyword and a low BMI”, or “a masculine tone with a tall height”.

For `combo_keyword_bmi`, we first categorized BMI values into 6 distinct labels:

- 0: BMI between 0–10, indicating invalid weight values
- 1: BMI between 10–18.5
- 2: BMI between 18.5–22
- 3: BMI between 22–25
- 4: BMI between 25–30
- 5: BMI above 30, indicating extremely overweight and invalid height values

We then combined the keyword label (which could be 0, 1, or 2) with the BMI label. For example, if a sample contains the word “handsome” (keyword = 1) and has a BMI of 23 (BMI label = 3), the `combo_keyword_bmi` value would be 1_3.

Similarly, the `combo_keyword_height` feature was created by combining the keyword label with the height bin of each sample. Height was categorized into six labels based on common sense and the distribution in the training dataset:

- 0: Height below 130 cm, representing invalid height values
- 1: Height between 130–150 cm
- 2: Height between 150–160 cm
- 3: Height between 160–170 cm
- 4: Height between 170–180 cm
- 5: Height between 180–200 cm
- 6: Height above 200 cm, representing extremely tall and invalid height values

We then combined the keyword label with the height bin label. For example, if a sample contains the word “cute” (keyword = 2) and a height of 165 cm (height label = 3), the `combo_keyword_height` value would be 2_3.

3. METHODS

3.1 FEATURE ENGINEERING

We chose XGBoost as our model to perform the given task. Before training our model, we performed several preprocessing steps to prepare the data for XGBoost. First, we encoded the gender label from values 1 (boy) and 2 (girl) into 0 and 1 to ensure compatibility with XGBoost. We then calculated the scale positive class weight to address the class imbalance issue in the training dataset, which

contains 316 male and 107 female samples. This weight was passed to the model to ensure that girl samples received greater attention during training. Afterwards, we converted all object-type columns into category types, enabling native categorical feature support via `enable_categorical=True` in XGBoost.

3.1.1 General Method

To systematically identify the best combination of features and model parameters, we implemented several procedures that perform feature selection, hyperparameter tuning, and model evaluation with cross-validation.

Firstly, we trained an initial XGBoost model using all the preprocessed features. We then ranked all features in descending order based on their importance scores, which were obtained from the XGBoost model's `feature_importances_` attribute.

Secondly, we applied a stepwise selection strategy. Starting with the top 3 features, we progressively added one feature at a time and evaluated the performance of each feature subset using 10-fold cross-validation. This approach allowed us to identify the subset of features that achieved the highest accuracy.

Third, we defined a search range for several key hyperparameters and used grid search to find the optimal combination with the best accuracy. The hyperparameters included: `learning_rate`, `max_depth`, and `reg_alpha` and `reg_lambda` for preventing overfitting.

Lastly, we used the best-performing parameter set to train the XGBoost model on the entire training set. To further prevent overfitting and determine the optimal number of boosting rounds, we plotted the training log loss across all iterations and identified the round at which the loss reached its minimum. This optimal number of rounds was then used—together with the selected parameters—to retrain the final model.

3.1.2 Recursive Feature Elimination (RFE)

Recursive Feature Elimination (RFE) is a widely used feature selection technique classified as a wrapper method. Its main goal is to iteratively remove the least important features to improve model performance and interpretability. RFE requires a base machine learning model to evaluate feature importance during each iteration. In various classification tasks across different fields, RFE has been shown to enhance accuracy and generalization, while also reducing the risk of overfitting [2], [3], [4], [5]. In this study, we applied RFE in combination with XGBoost to identify and rank the most important features.

3.2 Model Design

3.2.1 XGBoost

XGBoost is an abbreviation of eXtreme Gradient Boost. It is an ensemble learning algorithms and an improved version of the Gradient Boosted Decision Tree (GBDT) algorithm [6].

The core concept of XGBoost is to utilize gradient boosting to learn the residuals from the previous tree continuously and apply gradient descent to minimize the loss function. This enables the new tree to focus on reducing the residuals during training, finally improving accuracy. Moreover, XGBoost adds regularization in order to mitigate the risk of over-fitting [7], [8].

3.2.2 K-folds Cross Validation

K-fold cross-validation is a widely used model evaluation method in statistics and machine learning. It aims to assess a model's generalization ability and reduce the bias caused by relying on a single train/validation split. The main idea is to divide the dataset evenly into K subsets (folds) and perform K rounds of training and evaluation. In each round, one-fold is used as the validation set

while the remaining $K-1$ folds are used for training. A model is trained and evaluated in each round, and the final performance is calculated as the average across all K rounds [9].

One important consideration is when dealing with imbalanced datasets. In such cases, it is necessary to use the *stratify* parameter during the split to ensure that the class distribution remains consistent across all folds [10]. Although K -fold cross-validation helps reduce overfitting, it can be computationally expensive for large datasets. In this study, however, the dataset is relatively small, so a 10-fold cross-validation is used without concerns about computational cost. Nonetheless, it is still important to check whether each fold maintains a balanced class distribution.

3.3 Evaluation metrics

3.3.1 Accuracy

In this study, we use 10-fold cross-validation to evaluate model performance. For each selected number of top features, we perform cross-validation and compute the average accuracy across the 10-folds. The average accuracy serves as the evaluation metric. We then select the number of features that yield the highest average accuracy as the optimal feature combination for the final model.

Accuracy is calculated as the number of correctly predicted samples divided by the total number of samples:

$$\text{Accuracy} = \frac{\text{The number of samples predicted correctly}}{\text{Total number of samples}}$$

In this study, we report the average accuracy obtained from 10-fold cross-validation as the evaluation metric.

3.3.2 Loss Curve

To monitor the model's convergence and generalization performance during training, we plot the loss curve based on the logarithmic loss (*log loss*) values recorded at each boosting iteration. The log loss is used as the evaluation metric (*eval_metric='logloss'*) in XGBoost, and is calculated as follows:

$$\text{Logloss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where $y_i \in \{0, 1\}$ denotes the true label, and $p_i \in [0, 1]$ is the predicted probability of the positive class. The loss is computed separately for the training and validation sets in each iteration.

The training loss curve reflects how well the model fits the training data, while the validation loss curve indicates generalization to unseen data. A steadily decreasing validation loss implies proper learning and model convergence. Conversely, a divergence between training and validation losses may indicate overfitting.

4. Experiment Design

We experimented with several different approaches to tackle this Kaggle prediction task. Below is a summary of the experimental designs behind our two best-performing submissions, both achieved a final accuracy of 86.38% on the leaderboard.

4.1 Experiment 1

This method addresses all the data preprocessing methods mentioned above, only except the KNN imputation for the training and testing dataset. For the feature engineering part, it only addresses the "general method" as mentioned above in 3.1.1, not using the RFE to identify the important features. As for the model training, the best hyperparameters set found were listed below: *learning_rate*: 0.27, *max_depth*: 5, *reg_alpha*: 0.17, *reg_lambda*: 2.6 and best round: 13.

The major difference of this method is the way dealing with missing and extreme value in the testing dataset. As it replaced missing and extreme value in the training dataset based on the gender's average, there is no way to know the gender in the testing dataset beforehand. To address this problem, it follows the following steps, while the average height, weight value and outlier boundary were already calculated from the training dataset:

- (1) **Sort the testing dataset in descending order** based on the height column.
- (2) **Convert negative values to positive:** If any height or weight values were negative, we first converted them to their absolute values.
- (3) **If both height and weight are missing:** Fill in both values using the average height and weight of the entire training dataset.
- (4) **If both height and weight are extreme values (invalid):** Assign value 1 to both fields as a flag, since no reliable estimation could be made.
- (5) **If only height is missing or invalid:** Estimate the height value using the average_BMI of the entire training dataset.
- (6) **If only weight is missing or invalid:** Fill in the missing value by taking the average weight of the six nearest neighbors (three samples with higher and three with lower height values).

The rationale behind this design is the assumption that individuals with similar height tend to have similar weight. However, we acknowledge that this approach may not be entirely appropriate and could potentially introduce data leakage, as it makes use of local neighborhood information within the test set.

4.2 Experiment 2

The experiment begins with the training and testing datasets, both of which undergo a series of preprocessing steps. Initially, a subset of relevant features is selected for downstream processing, specifically: *id*, *gender*, *height*, *weight*, and *self_intro*. For the numerical features height and weight, outliers are detected using the interquartile range (IQR) method. Rather than discarding these outlier observations, values falling outside the IQR bounds are replaced with missing values to retain the full dataset while minimizing the influence of extreme values.

The textual feature *self_intro* is then subjected to standard text cleaning procedures to remove irrelevant symbols, punctuation, and noise. Following this, missing values in numerical features (height, weight) are imputed using k-nearest neighbors (KNN) imputation, which leverages the similarity between samples to estimate plausible values.

Subsequent to data cleaning and imputation, new features (*self_intro_len*, *self_intro_words*, and *keywords*) are engineered from the *self_intro* field. The process of adding new features is same as section 2.3.

For feature engineering, Recursive Feature Elimination (RFE) with XGBoost is employed to obtain a ranked list of features based on importance scores. Using this ranking, a wrapper-based evaluation strategy is implemented in which features are incrementally added (from most to least important), and a model is trained and evaluated via 10-fold cross-validation at each step. The average cross-validation accuracy is recorded for each subset.

Finally, the feature subset yielding the highest average cross-validation performance is identified as the optimal set. The corresponding XGBoost model is saved and used to generate

predictions on the test set. The results are exported in CSV format for subsequent evaluation.

5. REFERENCES

- [1] T. Emmanuel, T. Maupong, D. Mpoeleng, T. Semong, B. Mphago, and O. Tabona, "A survey on missing data in machine learning," *J. Big Data*, vol. 8, no. 1, p. 140, 2021, doi: 10.1186/s40537-021-00516-9.
- [2] M. Awad and S. Fraihat, "Recursive Feature Elimination with Cross-Validation with Decision Tree: Feature Selection Method for Machine Learning-Based Intrusion Detection Systems," *J. Sens. Actuator Netw.*, vol. 12, no. 5, Art. no. 5, Oct. 2023, doi: 10.3390/jsan12050067.
- [3] A. R. Barzani, P. Pahlavani, O. Ghorbanzadeh, K. Gholamnia, and P. Ghamisi, "Evaluating the Impact of Recursive Feature Elimination on Machine Learning Models for Predicting Forest Fire-Prone Zones," *Fire*, vol. 7, no. 12, Art. no. 12, Dec. 2024, doi: 10.3390/fire7120440.
- [4] G. S. Fuhnwi, M. Revelle, and C. Izurieta, "Improving Network Intrusion Detection Performance: An Empirical Evaluation Using Extreme Gradient Boosting (XGBoost) with Recursive Feature Elimination," in *2024 IEEE 3rd International Conference on AI in Cybersecurity (ICAIC)*, Feb. 2024, pp. 1–8. doi: 10.1109/ICAIC60265.2024.10433805.
- [5] M. Kumar, "Early detection of chronic kidney disease using recursive feature elimination and cross-validated XGBoost model," *Int. J. Comput. Mater. Sci. Eng.*, Aug. 2023, doi: 10.1142/S2047684123500367.
- [6] S. S. Azmi and S. Baliga, "An Overview of Boosting Decision Tree Algorithms utilizing AdaBoost and XGBoost Boosting strategies," vol. 07, no. 05, 2020.
- [7] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco California USA: ACM, Aug. 2016, pp. 785–794. doi: 10.1145/2939672.2939785.
- [8] T. R. Mahesh, V. Vinoth Kumar, V. Muthukumaran, H. K. Shashikala, B. Swapna, and S. Guluwadi, "Performance Analysis of XGBoost Ensemble Methods for Survivability with the Classification of Breast Cancer," *J. Sens.*, vol. 2022, pp. 1–8, Sep. 2022, doi: 10.1155/2022/4649510.
- [9] S. Arlot and A. Celisse, "A survey of cross-validation procedures for model selection," *Stat. Surv.*, vol. 4, no. none, pp. 40–79, Jan. 2010, doi: 10.1214/09-SS054.
- [10] "3.1. Cross-validation: evaluating estimator performance," scikit-learn. Accessed: Apr. 07, 2025. [Online]. Available: https://scikit-learn/stable/modules/cross_validation.html

6. WORK DIVISION

Name	Work description	Contribution
Yun-Ting Lai 113423040	Coding: - Data preprocessing - Experiment 1 Document: - Abstract - Introduction - Data preprocessing (2.1) - Methods (3.1, 3.1.1) - Experiment Design (4.1)	50%
Wei-Siang Chen 113423052	Coding - KNN imputation - Experiment 2 Document - Data preprocessing (2.2) - Methods (3.1.2, 3.2) - Experiment Design (4.2)	50%